

Java Constructors

A. Introduction to Java Constructors

A constructor is a special member of a class that is automatically executed when an object is created. The primary purpose of a constructor is to initialize the object's state.

Consider the following Student class:

```
public class Student {  
    String name;  
    int age;  
}
```

When an object is created, its instance variables initially contain Java's default values.

```
Student student = new Student();
```

The initial values are:

- name = null
- age = 0

We can manually assign values, but a constructor lets us initialize the object immediately.

```
public class Student {  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
Student student = new Student("Juan", 20);
```

The general syntax of a constructor is:

```
ClassName(parameters) {  
    // initialization statements  
}
```

B. Rules for Creating a Java Constructor

Rule 1: The constructor name must be the same as the class name.

Correct:

```
class Student {  
  
    Student() {  
        System.out.println("Student created");  
    }  
}
```

Incorrect:

```
class Student {  
  
    student() {  
        // Incorrect constructor name  
    }  
}
```

Rule 2: A constructor does not have a return type.

Correct:

```
class Student {  
  
    Student() {  
        System.out.println("Constructor executed");  
    }  
}
```

Incorrect:

```
class Student {  
  
    void Student() {  
        System.out.println("This is actually a method");  
    }  
}
```

The second example is not a constructor. Because it has void, Java treats it as a method named Student.

Rule 3: Constructors are automatically called when an object is created.

```
class Student {  
  
    Student() {  
        System.out.println("Constructor executed!");  
    }  
}
```

DCIT 50 – Object Oriented Programming

```
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
    }  
}
```

Output:

```
Constructor executed!
```

Rule 4: Constructors are not inherited.

```
class Person {  
  
    Person() {  
        System.out.println("Person constructor");  
    }  
}  
  
class Student extends Person {  
}
```

The constructor of Person is not inherited by Student. However, when a Student object is created, Java may invoke the superclass constructor through `super()`.

Rule 5: A class can have multiple constructors.

```
class Student {  
  
    Student() {  
    }  
  
    Student(String name) {  
    }  
  
    Student(String name, int age) {  
    }  
}
```

This is called constructor overloading.

Rule 6: If no constructor is declared, Java may provide a default constructor.

```
class Student {  
  
    String name;  
    int age;  
}
```

The compiler effectively provides a no-argument constructor, allowing:

```
Student s = new Student();
```

DCIT 50 – Object Oriented Programming

However, if a constructor is explicitly declared, Java no longer supplies the compiler-generated no-argument constructor automatically.

```
class Student {  
  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
}
```

This will cause an error:

```
Student s = new Student();
```

The correct form is:

```
Student s = new Student("Juan");
```

C. Types of Java Constructors

For introductory Java, constructors can be discussed in three useful categories:

- Default Constructor
- No-Argument Constructor
- Parameterized Constructor

1. Default Constructor

A default constructor is the constructor automatically provided by the Java compiler when no constructor is declared.

```
class Student {  
  
    String name;  
    int age;  
}
```

Conceptually, Java provides:

```
Student() {  
}  
  
Student student = new Student();
```

The fields receive Java's default values: name → null and age → 0.

2. No-Argument Constructor

A no-argument constructor is a constructor explicitly written by the programmer that has no parameters.

```
class Student {  
  
    String name;  
    int age;  
  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
}  
  
Student student = new Student();  
  
System.out.println(student.name);  
System.out.println(student.age);
```

Output:

```
Unknown  
0
```

Default Constructor vs. No-Argument Constructor

Default Constructor	No-Argument Constructor
<i>Usually refers to compiler-provided constructor</i>	<i>Explicitly written by programmer</i>
<i>Automatically generated if no constructor exists</i>	<i>Written manually</i>
<i>Usually has no parameters</i>	<i>Has no parameters</i>
<i>Performs default initialization</i>	<i>Can perform custom initialization</i>

3. Parameterized Constructor

A parameterized constructor accepts values when an object is created.

```
class Student {  
  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}  
  
Student student = new Student("Maria", 19);  
  
System.out.println(student.name);  
System.out.println(student.age);
```

Output:

```
Maria  
19
```

Without a constructor, the values would have to be assigned separately. With a parameterized constructor, object creation and initialization can happen in one statement.

D. Understanding the this Keyword in Constructors

The this keyword refers to the current object.

```
class Student {  
  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

In `this.name = name`, the left side refers to the instance variable and the right side refers to the constructor parameter.

Therefore, `this.name = name` means: current object's name = parameter name.

E. Constructor Overloading

Constructor overloading occurs when a class has multiple constructors with different parameter lists.

```
class Student {  
  
    String name;  
    int age;  
    String course;  
  
    Student() {  
        name = "Unknown";  
        age = 0;  
        course = "Unknown";  
    }  
  
    Student(String name) {  
        this.name = name;  
        age = 0;  
        course = "Unknown";  
    }  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
        course = "Unknown";  
    }  
  
    Student(String name, int age, String course) {  
        this.name = name;  
        this.age = age;  
        this.course = course;  
    }  
}
```

Objects can now be created in different ways:

```
Student s1 = new Student();  
Student s2 = new Student("Juan");  
Student s3 = new Student("Maria", 20);  
Student s4 = new Student("Pedro", 21, "BSIT");
```

Java determines which constructor to execute based on the arguments supplied.

Constructor Overloading Rules

The constructors must have different parameter lists.

Valid:

```
Student()  
Student(String name)  
Student(String name, int age)
```

Also valid:

DCIT 50 – Object Oriented Programming

```
Student(String name)  
Student(int age)
```

Invalid:

```
Student(String name)  
Student(String studentName)
```

These are considered the same parameter list because both accept one String. Parameter names do not determine overloading.

F. Constructor Chaining

When several constructors perform similar initialization, one constructor can call another constructor using `this(...)`.

```
class Student {  
  
    String name;  
    int age;  
    String course;  
  
    Student() {  
        this("Unknown", 0, "Unknown");  
    }  
  
    Student(String name) {  
        this(name, 0, "Unknown");  
    }  
  
    Student(String name, int age) {  
        this(name, age, "Unknown");  
    }  
  
    Student(String name, int age, String course) {  
        this.name = name;  
        this.age = age;  
        this.course = course;  
    }  
}
```

For example, `new Student()` calls the no-argument constructor, which then calls the three-parameter constructor.

Important rule: a call to `this(...)` must be the first statement inside a constructor.

Correct:

```
Student() {  
    this("Unknown");  
}
```

Incorrect:

```
Student() {  
    System.out.println("Creating student");  
    this("Unknown"); // Error  
}
```

G. Java Copy Constructor

Java does not have a special built-in copy constructor feature like C++, but a programmer can create a constructor that receives an object of the same class and copies its values. This is commonly called a copy constructor.

```
class Student {  
  
    String name;  
    int age;  
    String course;  
  
    Student(String name, int age, String course) {  
        this.name = name;  
        this.age = age;  
        this.course = course;  
    }  
  
    Student(Student other) {  
        this.name = other.name;  
        this.age = other.age;  
        this.course = other.course;  
    }  
}  
  
Student original =  
    new Student("Juan", 20, "BSIT");  
  
Student copy =  
    new Student(original);
```

The copy contains the same field values:

Original	Copy
-----	-----
name = Juan	name = Juan
age = 20	age = 20
course = BSIT	course = BSIT

Copy Constructor vs. Assignment

Students often confuse object assignment with copying.

Assignment:

```
Student copy = original;
```

Both variables refer to the same object.

Copy constructor:

```
Student copy = new Student(original);
```

This creates a new Student object containing copied values. This distinction becomes important when studying object references.

H. Constructor vs. Method

```
class Student {  
  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
  
    void displayName() {  
        System.out.println(name);  
    }  
}
```

Student(String name) is a constructor, while displayName() is a method.

Constructor	Method
Initializes an object	Performs an operation
Must have the same name as the class	Can have any valid name
Cannot have a return type	May have a return type
Automatically invoked during object creation	Usually explicitly invoked
Cannot be inherited	Methods can be inherited
Cannot be overridden	Methods can be overridden
Establishes initial object state	Manipulates or accesses object behavior

Example:

```
class Student {  
  
    String name;  
    int age;  
  
    // Constructor  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // Method  
    void introduce() {  
        System.out.println("My name is " + name);  
        System.out.println("I am " + age + " years old.");  
    }  
}  
  
Student student = new Student("Maria", 20);  
  
student.introduce();
```

The constructor is used during object creation and initialization, while the method is called afterward to perform an operation.

I. Complete Example

The following program combines no-argument, parameterized, copy constructor, constructor overloading, and methods.

```
class Student {  
  
    private String name;  
    private int age;  
    private String course;  
  
    // No-argument constructor  
    Student() {  
        this("Unknown", 0, "Unknown");  
    }  
  
    // Parameterized constructor  
    Student(String name, int age, String course) {  
        this.name = name;  
        this.age = age;  
        this.course = course;  
    }  
  
    // Copy constructor  
    Student(Student other) {  
        this.name = other.name;  
        this.age = other.age;  
        this.course = other.course;  
    }  
  
    // Method  
    void displayInformation() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
        System.out.println("Course: " + course);  
        System.out.println();  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        // No-argument constructor  
        Student student1 = new Student();  
  
        // Parameterized constructor  
        Student student2 =  
            new Student("Juan", 20, "BSIT");  
  
        // Copy constructor  
        Student student3 =  
            new Student(student2);  
  
        student1.displayInformation();  
        student2.displayInformation();  
        student3.displayInformation();  
    }  
}
```

DCIT 50 – Object Oriented Programming

```
}  
}
```

Output:

```
Name: Unknown  
Age: 0  
Course: Unknown
```

```
Name: Juan  
Age: 20  
Course: BSIT
```

```
Name: Juan  
Age: 20  
Course: BSIT
```